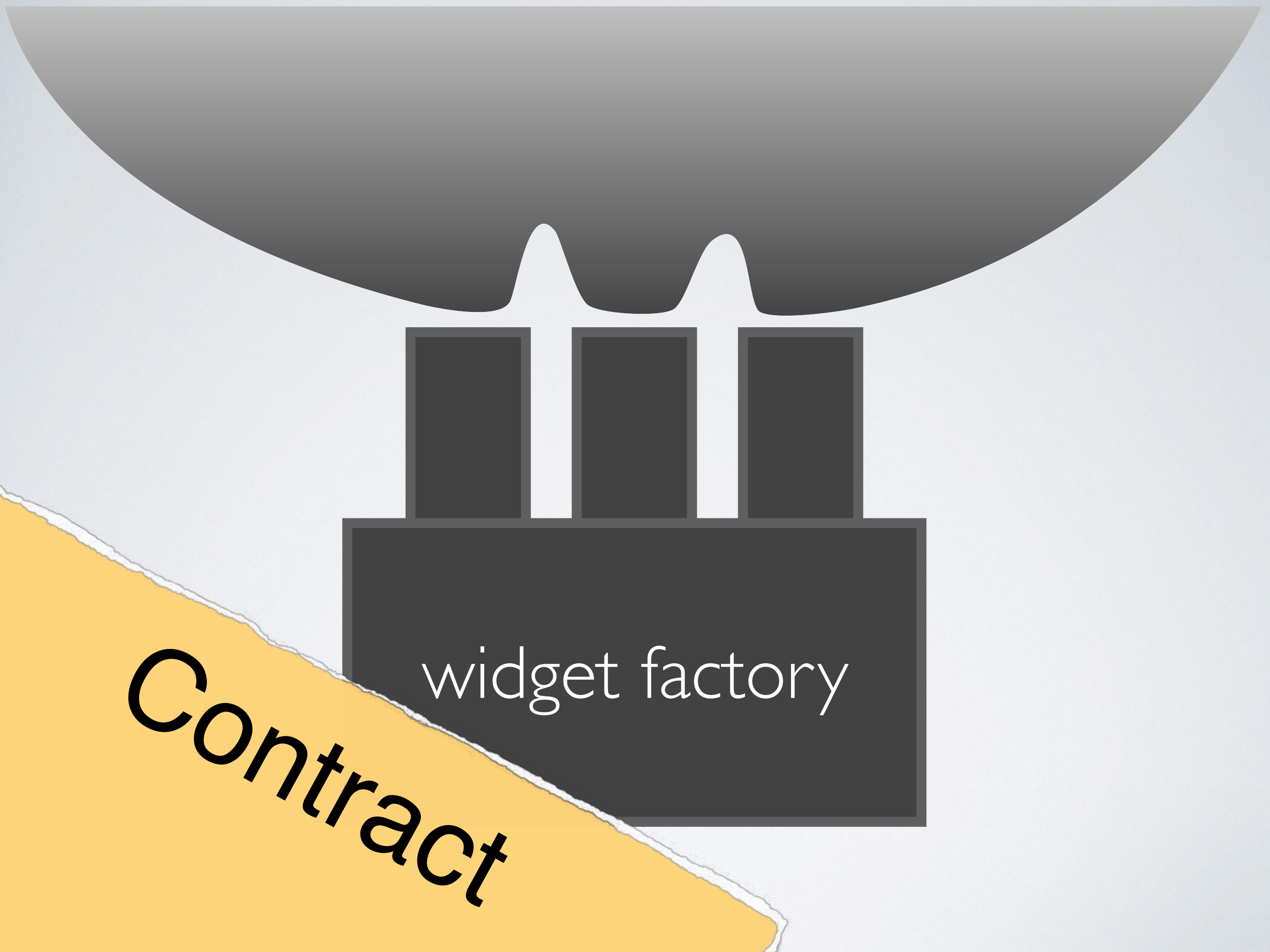


Visual Code Annotations for Cyberphysical Programming

Ben Swift, Henry Gardner, John Hosking
Research School of Computer Science
Australian National University

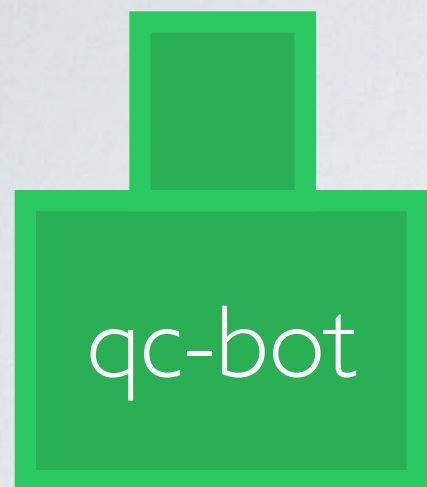
Andrew Sorensen
Institute for Future Environments,
Queensland University of Technology



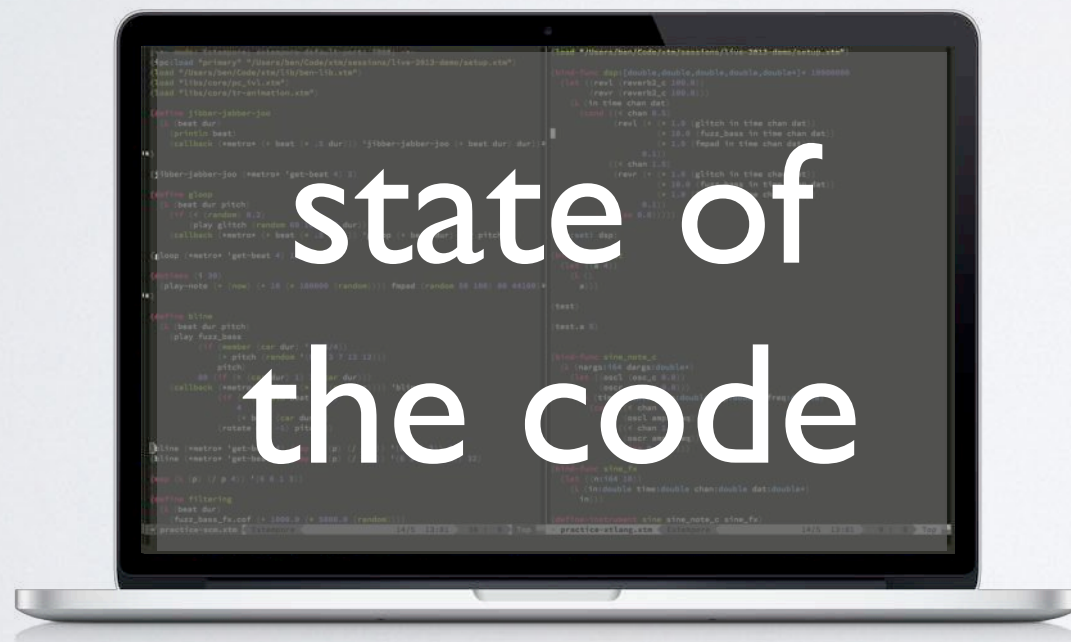


widget factory

Contract



cyberphysical programming



the cyberphysical programmer



Extempore



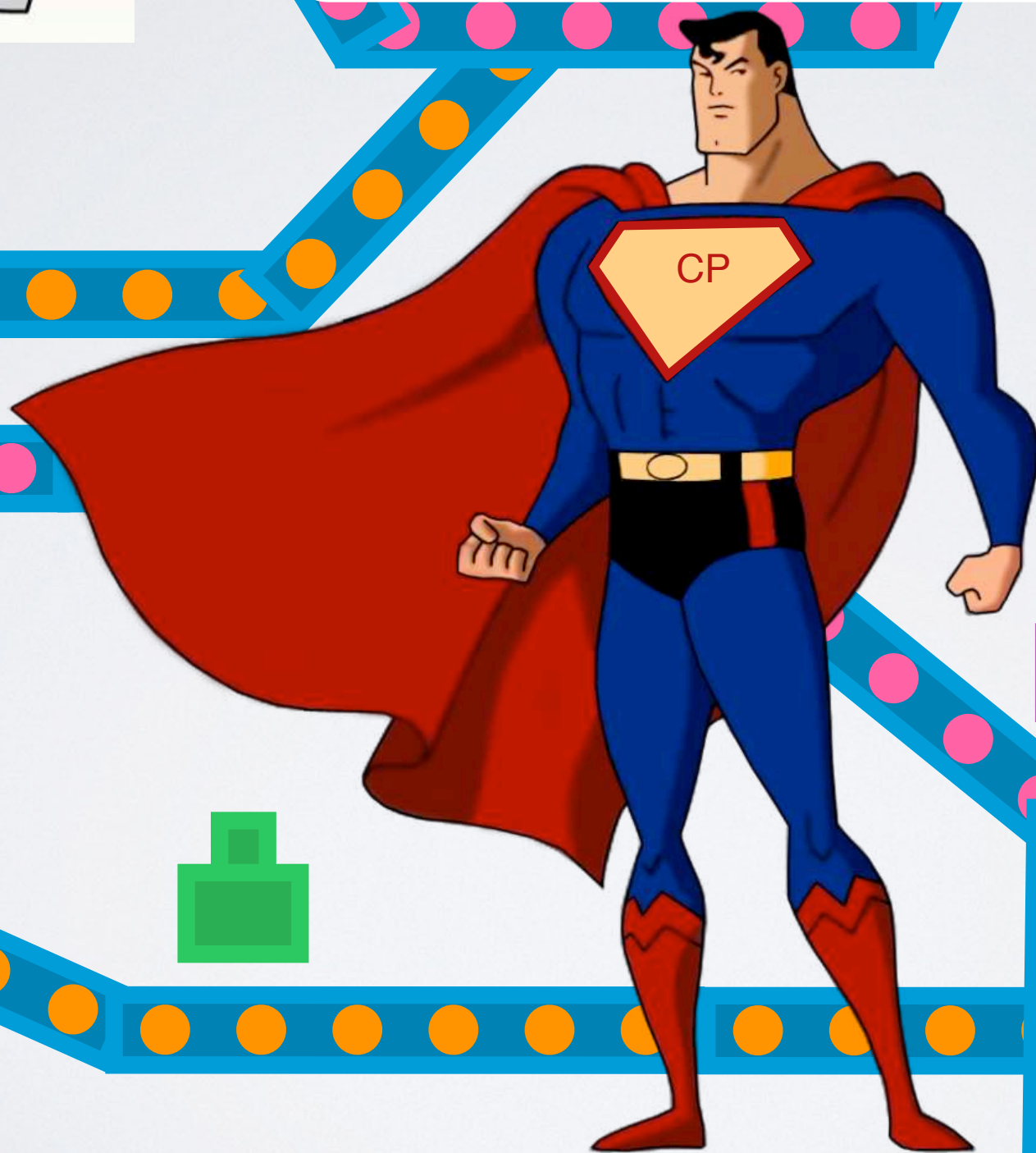
extempore.moso.com.au
github.com/digego/extempore







*“I need our quality control
bots online—**now!**”*





*“I need our quality control bots online—**now!**”*

```
;; cyberphysical programming in the widget factory
```

```
## (c) 2010-2012 ##  
##  
#####  
#####  
#####  
#####  
#####  
#####
```

---PortAudio---

```
Output Device : Built-in Output  
Input Device :  
SampleRate : 44100  
Channels Out : 2  
Channels In : 0  
Frames : 128  
Latency : 0.0038322
```

Starting **primary** process

Trying to connect to 'localhost' on port 7099

New Client Connection

Successfully connected to remote process

Starting **utility** process

Trying to connect to 'localhost' on port 7098

New Client Connection

Successfully connected to remote process

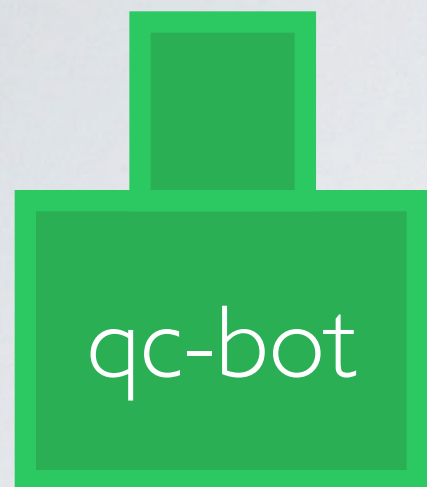
New Client Connection

Starting OSC server on port: 7009 calling back to TR-anim-cl
ient-port

New Client Connection

Attempting to return a string from non-symbol obj ()

Trace: callback-adapter



visual code annotations

- code-centric (inline)
- passive
- provide context about the *relationship* between the **code** and the **world**



“We can’t stop production for **one** overweight sample!”

```
;; cyberphysical programming in the widget factory
```

```
(define qc-weight-check
  (let ((strikes))
    (lambda (time delta-t)
      (let ((w (sample-weight 1)))
        (if (> w 0.999)
            (stop-production "overweight sample")
            (callback (+ time (* *second* delta-t) 0.
5)

              'qc-weight-check
              (+ time (* *second* delta-t))
              delta-t))))))
```

```
(qc-weight-check (now) 1)
```

```
##
#####
#####
#####
#####
##
```

---PortAudio---

Output Device : Built-in Output
Input Device :
SampleRate : 44100
Channels Out : 2
Channels In : 0
Frames : 128
Latency : 0.0038322

Starting primary process

Trying to connect to 'localhost' on port 7099

New Client Connection

Successfully connected to remote process

Starting utility process

Trying to connect to 'localhost' on port 7098

New Client Connection

Successfully connected to remote process

New Client Connection

Starting OSC server on port: 7009 calling back to TR-anim-cl
ient-port

New Client Connection

Attempting to return a string from non-symbol obj ()

Trace: callback-adapter

Stopping production due to overweight sample

particularly helpful when dealing with multiple concurrent 'processes' running in the world

```
(define qc-weight-check
  (let ((strikes 0))
    (lambda (time delta-t)
      (let ((w (sample-weight 1)))
        (if (> w 0.9)
            (begin (set! strikes (+ strikes 1))
                    (println 'strikes= strikes)))
            (if (> strikes 20)
                (println "Overweight sample, stopping..."
                        'weight= w)
                (callback (+ time (* *second* delta-t) 0.5)
                          'qc-weight-check
                          (+ time (* *second* delta-t))
                          (* 2 (- 1 w))))))))
```

```
(qc-weight-check (now) 1)
```

```
(define qc-size-check
  (lambda (time delta-t)
    (if (< (random) 0.1)
        (println "Warning: size problem"))
    (callback (+ time (* *second* delta-t) 0.5)
              'qc-size-check
              (+ time (* *second* delta-t))
              delta-t))))
```

```
(qc-size-check (now) 0.5)
```

```
(define qc-colour-check
  (lambda (time delta-t)
    (if (< (random) 0.1)
        (println "Warning: colour problem."))
    (callback (+ time (* *second* delta-t) 0.5)
```

```
strikes= 15
"Warning: colour problem."
strikes= 16
"Warning: colour problem."
strikes= 17
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
strikes= 18
strikes= 19
strikes= 20
"Warning: size problem"
strikes= 21
"Overweight sample, stopping..." weight= 0.943653
"Warning: size problem"
"Warning: colour problem."
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
strikes= 1
"Warning: size problem"
"Warning: size problem"
strikes= 2
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
"Warning: size problem"
strikes= 3
"Warning: colour problem."
strikes= 4
```


lots more to say, but

the main point

the cyberphysical programmer has a
complex (live) relationship to the world

visual code annotations may help them
'manage' this relationship

open questions

when are these annotations helpful?

which ones are most helpful?

how do we measure helpfulness?

cheers

ben.swift@anu.edu.au

<http://benswift.me>

<http://extempore.moso.com.au>

